

nDCG for Evaluation, Worked Out by Hand

The gold-standard ranking metric for evaluation sets — graded relevance, logarithmic position discount, normalised to $[0, 1]$ so scores average fairly across queries. Computed in full, then averaged over a multi-query eval set.

What this document does. It computes nDCG for two queries with graded relevance judgments and averages them into a single system score — the exact procedure behind RAG/retrieval leaderboards. It uses the alternative “exponential-gain” formula common in evaluation tooling, and contrasts nDCG with MRR and MAP. Numbers from Appendix A.

Companion: the retrieval-flavoured nDCG is #18; this evaluation-flavoured PDF stresses cross-query averaging and the gain conventions used by eval frameworks. It sits beside MRR (#29) and MAP (#30).

1 Why nDCG for an eval set

To rank *systems* (not just results), you need a per-query score that (a) uses graded relevance, (b) rewards good ordering, and (c) is normalised so an easy, relevance-rich query cannot dominate the average. nDCG is the metric that satisfies all three, which is why BEIR, MTEB, and most RAG retrieval leaderboards report it.

Intuition

nDCG = *gain*, *discounted* by position, *normalised* by the ideal. Gain lets a perfect hit outweigh a so-so one; the log discount makes earlier positions matter more; normalising by the best-possible score (IDCG) maps every query to $[0, 1]$ so they can be averaged into one comparable number. That last step is what makes nDCG a *leaderboard* metric rather than just a per-query diagnostic.

2 The formula (exponential-gain convention)

Evaluation tools commonly use the exponential-gain form, which rewards high grades more sharply than the linear form of #18:

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \boxed{\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}},$$

then average over the query set: $\overline{\text{nDCG}} = \frac{1}{Q} \sum_i \text{nDCG}_i$. The gain $2^{\text{rel}} - 1$ maps grades $0, 1, 2, 3 \rightarrow 0, 1, 3, 7$ — so a grade-3 result is worth far more than three grade-1 results, reflecting that one excellent answer beats several mediocre ones.

3 Worked by hand

Query 1. Graded relevances down the ranking $\text{rel} = (3, 2, 0, 1)$. Gains $2^{\text{rel}} - 1 = (7, 3, 0, 1)$:

i	rel_i	$2^{\text{rel}_i} - 1$	$\log_2(i + 1)$	term
1	3	7	1.0000	7.0000
2	2	3	1.5850	1.8928
3	0	0	2.0000	0.0000
4	1	1	2.3219	0.4307

$$\text{DCG}_1 = 7 + 1.8928 + 0 + 0.4307 = 9.3235.$$

Ideal order of the same grades is $(3, 2, 1, 0)$, gains $(7, 3, 1, 0)$:

$$\text{IDCG}_1 = \frac{7}{1} + \frac{3}{1.585} + \frac{1}{2} + \frac{0}{2.3219} = 7 + 1.8928 + 0.5 + 0 = 9.3928.$$

$$\text{nDCG}_1 = \frac{9.3235}{9.3928} = 0.9926.$$

Near-perfect: the only flaw is the grade-1 result at rank 4 instead of rank 3 (an irrelevant grade-0 wastes rank 3).

Query 2. $\text{rel} = (0, 2, 3, 1)$, gains $(0, 3, 7, 1)$:

$$\text{DCG}_2 = \frac{0}{1} + \frac{3}{1.585} + \frac{7}{2} + \frac{1}{2.3219} = 0 + 1.8928 + 3.5 + 0.4307 = 5.8235.$$

Same ideal as before, $\text{IDCG}_2 = 9.3928$, so

$$\text{nDCG}_2 = \frac{5.8235}{9.3928} = 0.6200.$$

Query 2 ranks poorly — it put a grade-0 first and the best result (grade 3) only third.

Average over the eval set.

$$\overline{\text{nDCG}} = \frac{0.9926 + 0.6200}{2} = \boxed{0.8063}.$$

Mini-example · why normalisation makes averaging fair

Both queries here happen to share the same grade multiset, so the same IDCG — but in a real eval set, one query might have three grade-3 documents (huge IDCG) and another just one grade-1 (tiny IDCG). Raw DCG would let the relevance-rich query dominate the mean purely by having more gain available. Dividing each by its own IDCG puts both on $[0, 1]$ first, so the average reflects *ranking quality*, not how much relevance each query happened to contain. That is the property MRR and raw DCG lack.

4 Choosing nDCG vs. MRR vs. MAP

- **nDCG** — graded relevance, full ranking, cross-query comparable. The most informative, at the cost of needing graded human judgments. The default for retrieval leaderboards.
- **MAP (#30)** — binary relevance, all relevants, rank-aware. Use when labels are yes/no.
- **MRR (#29)** — binary, first hit only. Use for known-item search.

Pick the cheapest metric whose assumptions match your labels: graded $\rightarrow$ nDCG, binary multi-answer $\rightarrow$ MAP, single-answer $\rightarrow$ MRR.

Equation cheat-sheet

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad \text{nDCG} = \frac{\text{DCG}}{\text{IDCG}}, \quad \overline{\text{nDCG}} = \frac{1}{Q} \sum_i \text{nDCG}_i$$

graded + position + normalised; averages fairly across queries (leaderboard metric)

Appendix A · Reference implementation

```
import numpy as np
def dcg(rels):
    # exponential-gain DCG
    return sum((2**g - 1) / np.log2(i + 2) for i, g in enumerate(rels))
def ndcg(rels):
    return dcg(rels) / dcg(sorted(rels, reverse=True))

ndcg([3, 2, 0, 1])          # 0.9926   (query 1)
ndcg([0, 2, 3, 1])          # 0.6200   (query 2)
np.mean([ndcg([3,2,0,1]), ndcg([0,2,3,1])]) # 0.8063 (mean nDCG)
```

Internal learning material. Numbers reproducible from the appendix code. v1.0